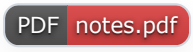
# Notes_25.11: Limitations of kNN & Curse of Dimensionality










🌐 **Language: English** | हिंदी / Hinglish | ⬅️ Lecture 25 Overview

---

## 📌 Executive Summary

Computational bottleneck, memory constraints, and the Curse of Dimensionality.

---

## 🎯 Mathematical Formulations

Distance Metric Contrast Collapse in High Dimensions:

$$ \lim_{d \to \infty} \frac{\text{dist}_{\max} - \text{dist}_{\min}}{\text{dist}_{\min}} \to 0 $$

Computational Complexity:

$$\text{Training Complexity: } \mathcal{O}(1), \qquad \text{Inference Latency: } \mathcal{O}(N \cdot d)$$

---

## 💡 Conceptual Principles & Architecture

### kNN Ki 3 Badi Kamzoriyan:

1. **Curse of Dimensionality:** Jaise-jaise features $d$ badhte hain, multi-dimensional space exponentially empty (sparse) ho jata hai. Sabhi points ke beech ka distance lagbhag barabar ho jata hai, jisse nearest neighbor ka concept hi bekar ho jata hai!

2. **Slow Inference:** Har nayi prediction ke liye model ko sabhi $N$ training points se distance calculate karna padta hai $\implies \mathcal{O}(N \cdot d)$. Agar 10 lakh rows hain, toh ek prediction mein seconds lag jayenge!
3. **Memory Hungry:** Pura dataset RAM mein rakhna padta hai.

---

## 💻 Python Production Implementation

```
# Simulating dimensionality curse
import numpy as np

dims = [2, 10, 50, 100, 500]
for d in dims:
    pts = np.random.uniform(0, 1, (100, d))
    dists = [np.linalg.norm(pts[0] - pts[i]) for i in range(1, 100)]
    ratio = (max(dists) - min(dists)) / min(dists)
    print(f"Dimension {d:3d} -> (Max-Min)/Min Distance Ratio: {ratio:.4f}")
```